

Ciência dos Dados I: Análise Exploratória

Aula 07: Controle de Fluxo, Funções Modulares e Boas Práticas

Prof. Dr. Ney Lemke

UNESP – Instituto de Biociências de Botucatu

Versão 2026

Sumário da Aula

Controle de Fluxo Idiomático

Desenvolvimento de Funções Modulares

Tratamento de Exceções

Laços Elegantes com zip e enumerate

Evite acessar elementos via índice manual `lista[i]`:

Iteração Paralela com zip

```
nomes = ['Ana', 'Beto']  
notas = [9.5, 8.0]  
for n, nota in zip(nomes, notas):  
    print(f"{n}: {nota}")
```

Contador com enumerate

```
for idx, nome in enumerate(nomes, start  
=1):  
    print(f"#{idx} - {nome}")
```

Anatomia de uma Função em Ciência de Dados

```
def calcula_estatisticas(amostras: list[float], ddof: int = 1) -> dict[str, float]:  
    """  
    Calcula a média e o desvio padrão de uma amostra numérica.  
  
    Parâmetros:  
        amostras: Lista de observações numéricas.  
        ddof: Graus de liberdade (padrão amostral = 1).  
    """  
    n = len(amostras)  
    media = sum(amostras) / n  
    var = sum((x - media)**2 for x in amostras) / (n - ddof)  
    return {'media': media, 'desvio_padrao': var**0.5}
```

Robustez com try / except

Pipelines analíticos processam dados reais que frequentemente contêm valores corrompidos:

```
valores_brutos = ['10.5', '20.3', 'ausente', '45.1', 'N/A']
valores_limpos = []
for v in valores_brutos:
    try:
        valores_limpos.append(float(v))
    except ValueError:
        pass # Ignora ou registra em log de auditoria
```

Resumo da Aula

- Funções puras e modulares transformam scripts soltos em pipelines reutilizáveis.
- Docstrings e validações evitam propagação silenciosa de erros de cálculo.
- **Prática:** Construção de funções estatísticas e tratamento defensivo de dados.